

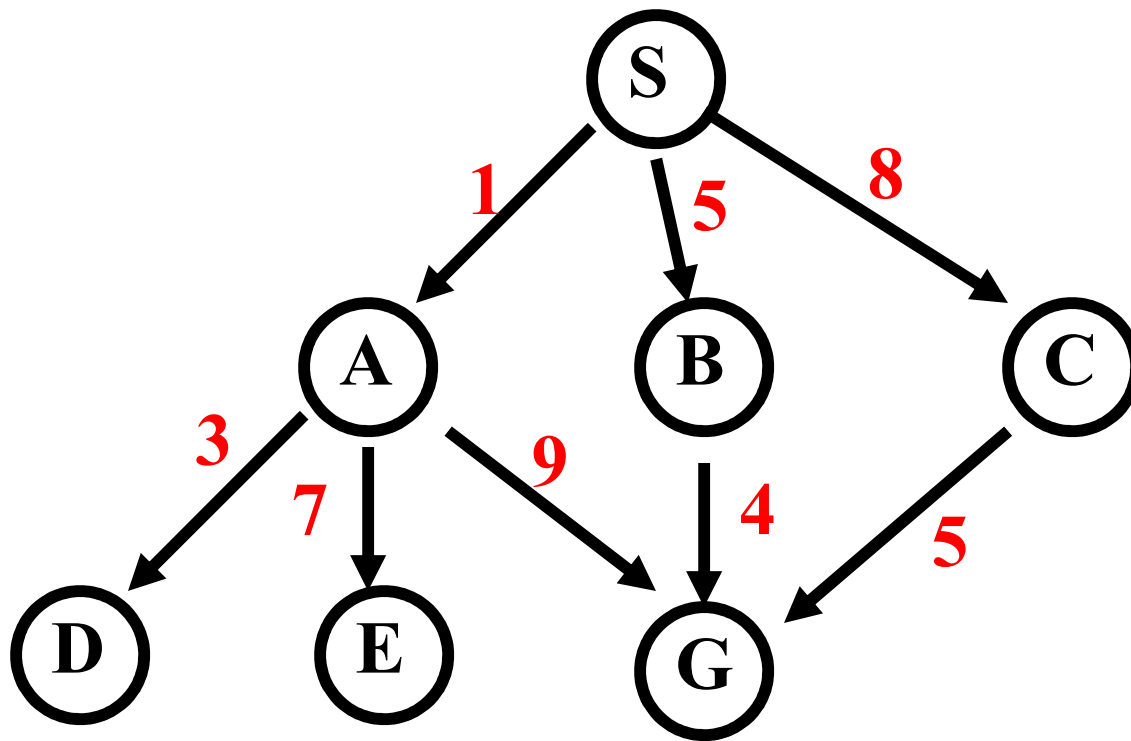
Uninformed vs. Informed Search

- **Uninformed Search Strategies**
 - Breadth-First search
 - Depth-First search
 - Uniform-Cost search
 - Depth Limited Search
 - Depth-First Iterative Deepening search
 - Bidirectional Search
- **Informed Search Strategies**
 - Greedy Best-first Search
 - A* Search
 - Hill climbing
 - Simulated Annealing
 - Beam search
 - Genetic Algorithm

Breadth-First

- Algorithm outline:
 - Always select from the OPEN the node with the **smallest depth(shallowest node)** for expansion, and put all newly generated nodes into **OPEN**
 - OPEN is organized as **FIFO** (first-in, first-out) list, i.e., a **queue**.
 - Terminate if a node selected for expansion is a goal

Example Illustrating Uninformed Search Strategies



Breadth-First Search

exp. node	OPEN list	CLOSED list
	{ S }	{ }
S	{ A B C }	{ S }
A	{ B C D E G }	{ S A }
B	{ C D E G G' }	{ S A B }
C	{ D E G G' G'' }	{ S A B C }
D	{ E G G' G'' }	{ S A B C D }
E	{ G G' G'' }	{ S A B C D E }
G	{ G' G'' }	{ S A B C D E }

Solution path found is S A G <-- this G also has cost 10
Number of nodes expanded (including goal node) = 7

Evaluating Search Strategies

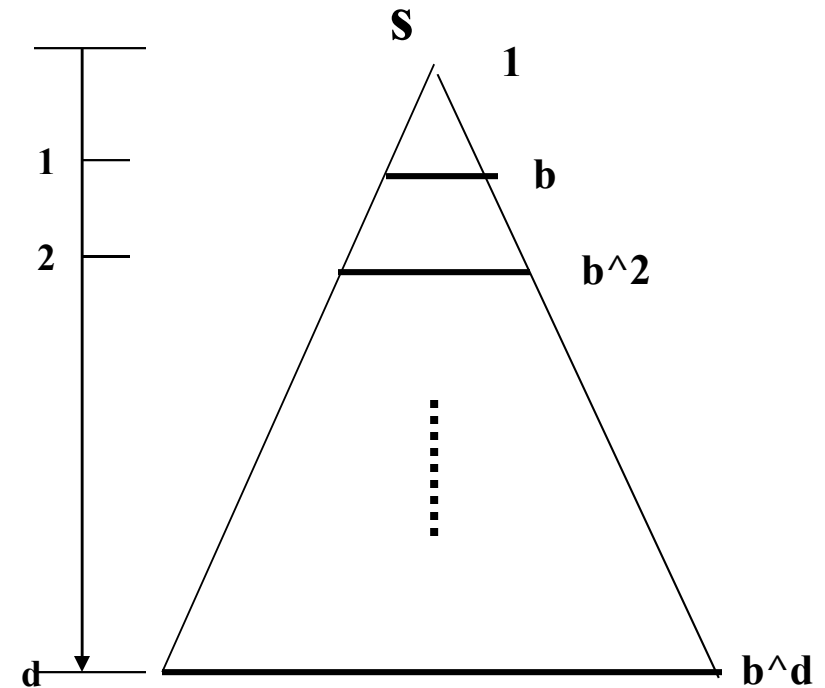
- **Completeness:** Whether the search strategy will find a solution if one exists.
- **Time Complexity:** How long (worst or average case) does it take to find a solution? Usually measured in terms of the **number of nodes expanded**.
- **Space Complexity:** How much space is used by the algorithm? Usually measured in terms of the **maximum size that the “OPEN” and “CLOSED” lists** becomes during the search
- **Optimality/Admissibility:** If a solution is found, is it guaranteed to be an optimal one? For example, is it the one with minimum cost?

Breadth-First Search (BFS)

- **Completeness:** BFS is complete if the branching factor is finite, meaning it will find a solution if one exists.
- **Time Complexity:** $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal node. This is because BFS explores all nodes at a given depth before moving to the next level.
- **Space Complexity:** $O(b^d)$, as it needs to store all generated nodes in the frontier, which can grow exponentially.
- **Optimality:** BFS is optimal if all path costs are the same, as it finds the shallowest solution first.

Breadth-First Search

- A complete search tree of depth d where each non-leaf node has b children, has a total of $1 + b + b^2 + \dots + b^d = (b^{d+1} - 1)/(b - 1)$ nodes
- Time complexity (# of nodes generated): $O(b^d)$
- Space complexity (maximum length of OPEN): $O(b^d)$

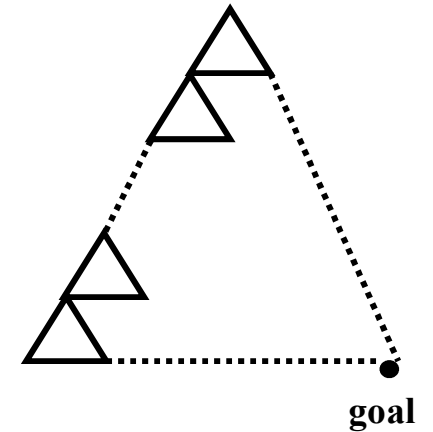


- For a complete search tree of depth 12, where every node at depths 0, ..., 11 has 10 children and every node at depth 12 has 0 children, there are $1 + 10 + 100 + 1000 + \dots + 10^{12} = (10^{13} - 1)/9 = O(10^{12})$ nodes in the complete search tree.

- **BFS is suitable for problems with shallow solutions**

Depth-First (DFS)

- Algorithm outline:
 - Always select from the OPEN the node with the **greatest depth** for expansion, and put all newly generated nodes into OPEN
 - OPEN is organized as **LIFO** (last-in, first-out) list.
 - Uses **Stack** Data structure.
 - Terminate if a node selected for expansion is a goal

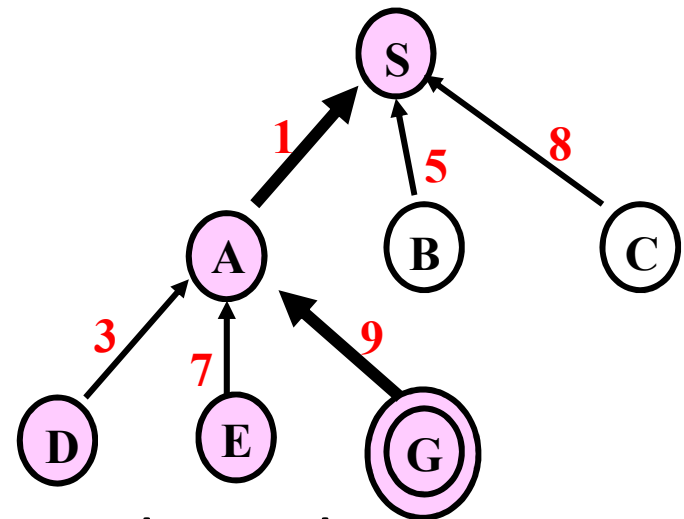


Depth-First Search

exp. node OPEN list

	{ S }
S	{ A B C }
A	{ D E G B C }
D	{ E G B C }
E	{ G B C }
G	{ B C }

CLOSED list



Solution path found is S A G <-- this G has cost 10

Number of nodes expanded (including goal node) = 5

Depth-First Search (DFS)

- **Completeness:** DFS is not complete in general because it may get stuck in an infinite depth if the search tree is infinite.
- **Time Complexity:** $O(b^m)$, where m is the maximum depth of the search tree. This is because DFS explores as far as possible down each branch before backtracking.
- **Space Complexity:** $O(bm)$, as it only needs to store a single path from the root to a leaf node along with remaining unexplored sibling nodes.
- **Optimality:** DFS is not optimal, as it may find a deeper (and potentially more costly) solution before finding a shallower one.

Uniform-Cost (UCS)

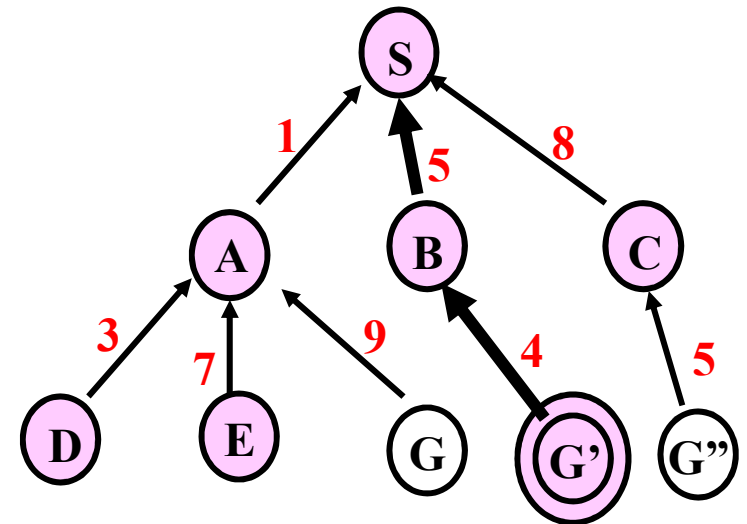
- Let $g(n)$ = cost of the path from the start node to an open node n .
- Algorithm outline:
 - Always select from the OPEN the node with the **least $g(.)$ value** for expansion, and put all newly generated nodes into OPEN
 - Nodes in OPEN are sorted by their $g(.)$ values (in ascending order).
 - It uses **Priority Queue**.
 - Terminate if a node selected for expansion is a goal.

Uniform-Cost Search

exp. node nodes list

	{S(0)}
S	{A(1) B(5) C(8)}
A	{D(4) B(5) C(8) E(8) G(10)}
D	{B(5) C(8) E(8) G(10)}
B	{C(8) E(8) G'(9) G(10)}
C	{E(8) G'(9) G(10) G''(13)}
E	{G'(9) G(10) G''(13)}
G'	{G(10) G''(13)}

CLOSED list



Solution path found is S B G <-- this G has cost 9, not 10
 Number of nodes expanded (including goal node) = 7

Uniform-Cost Search (UCS)

- **Completeness:** UCS is complete, as long as the cost of each step is greater than some small positive constant (i.e., no infinite paths with finite costs).
- **Time Complexity:** $O(b^d)$, similar to BFS in the worst case, but depends on the path costs.
- **Space Complexity:** $O(b^d)$, since it keeps all generated nodes in memory.
- **Optimality:** UCS is optimal if path costs are non-negative because it expands the least costly path first.

worst case: UCS becomes BFS when all arcs cost the same

Depth-Limited Search (DLS)

- Depth Limited Search is a modified version of DFS that **imposes a limit on the depth of the search.**
- This means that the algorithm will only explore nodes up to a certain depth, effectively preventing it from going down excessively deep paths that are unlikely to lead to the goal.
- By setting a maximum depth limit, DLS aims to improve efficiency and ensure more manageable search times.

Depth-Limited Search (DLS)

- **Completeness:** DLS is complete if the depth limit is greater than or equal to the depth of the shallowest goal. However, if the limit is too low, it may miss the solution.
- **Time Complexity:** $O(b^l)$, where l is the depth limit.
- **Space Complexity:** $O(bl)$, since it needs to store up to l nodes at the deepest level.
- **Optimality:** DLS is not optimal, as it might not find the least-cost solution if the limit is set too low.

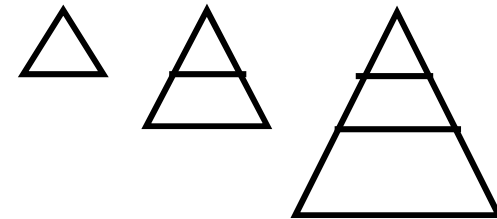
Depth-First Iterative Deepening Search (DFID)

- BF and DF both have exponential time complexity $O(b^d)$
 - BF is **complete** but has exponential space complexity (**conservative**)
 - DF has **linear space complexity** but is incomplete (**radical**)
- Space is often a **harder** resource constraint than time
- Can we have an algorithm that
 - Is complete
 - Has linear space complexity, and
 - Has time complexity of $O(b^d)$
- DFID by Korf in 1985
- First do DFS to depth 0 (i.e., treat start node as having no successors), then, if no solution found, do DFS to depth 1, etc.

until solution found do

DFS with depth bound c

$c = c+1$



Iterative Deepening Search (IDS)

- **Completeness:** IDS is complete as it incrementally increases the depth limit until it finds the goal.
- **Time Complexity:** $O(b^d)$, as it essentially performs a DFS up to each level d .
- **Space Complexity:** $O(bd)$, similar to DFS, as it only needs to store the current path from the root to the leaf node.
- **Optimality:** IDS is optimal if path costs are uniform, as it finds the shallowest solution.

Depth-First Iterative Deepening Search - Drawback

- If branching factor is b and solution is at depth d , then nodes at depth d are generated once, nodes at depth $d-1$ are generated twice, etc., and node at depth 1 is generated d times.

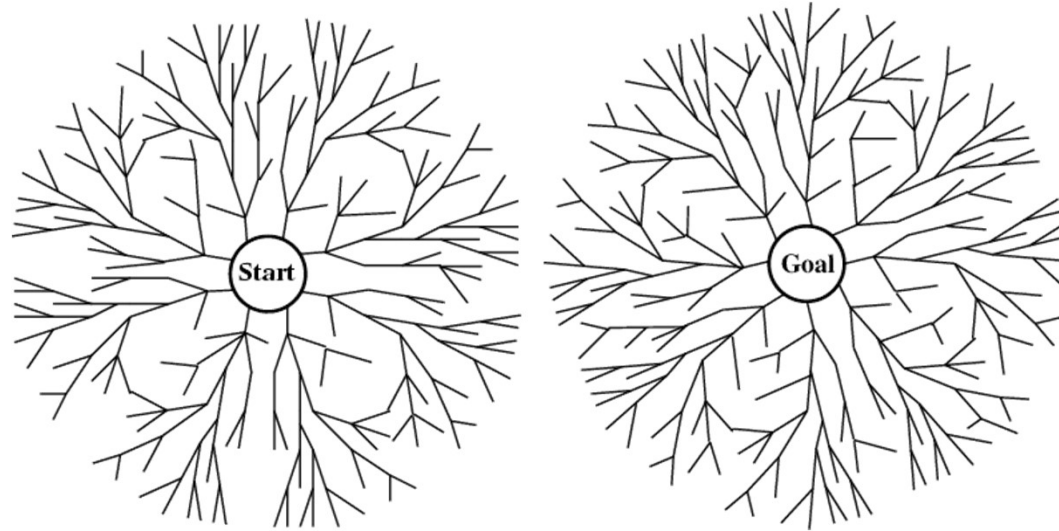
Hence

$$\text{total}(d) = b^d + 2b^{(d-1)} + \dots + db$$

$$\leq b^d / (1 - 1/b)^2 = O(b^d).$$

- If $b=4$, then worst case is $1.78 * 4^d$, i.e., 78% more nodes searched than exist at depth d (in the worst case).

Bi-directional search



- Alternate searching from the start state toward the goal and from the goal state toward the start.
- Stop when the frontiers intersect.
- Works well only when there are unique start and goal states and **when actions are reversible.**
- Can lead to finding a solution more quickly (but watch out for pathological situations).

Comparing Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Time	b^d	b^d	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	b^d	bm	bl	bd	$b^{d/2}$
Optimal?	Yes	Yes	No	No	Yes	Yes
Complete?	Yes	Yes	No	Yes, if $l \geq d$	Yes	Yes

Summary: Each strategy has its strengths and weaknesses, and the choice of which to use depends on the specific problem and its constraints.

When to use what

- **Depth-First Search:**

- Many solutions exist
- Know (or have a good estimate of) the depth of solution

- **Breadth-First Search:**

- Some solutions are known to be shallow

- **Uniform-Cost Search:**

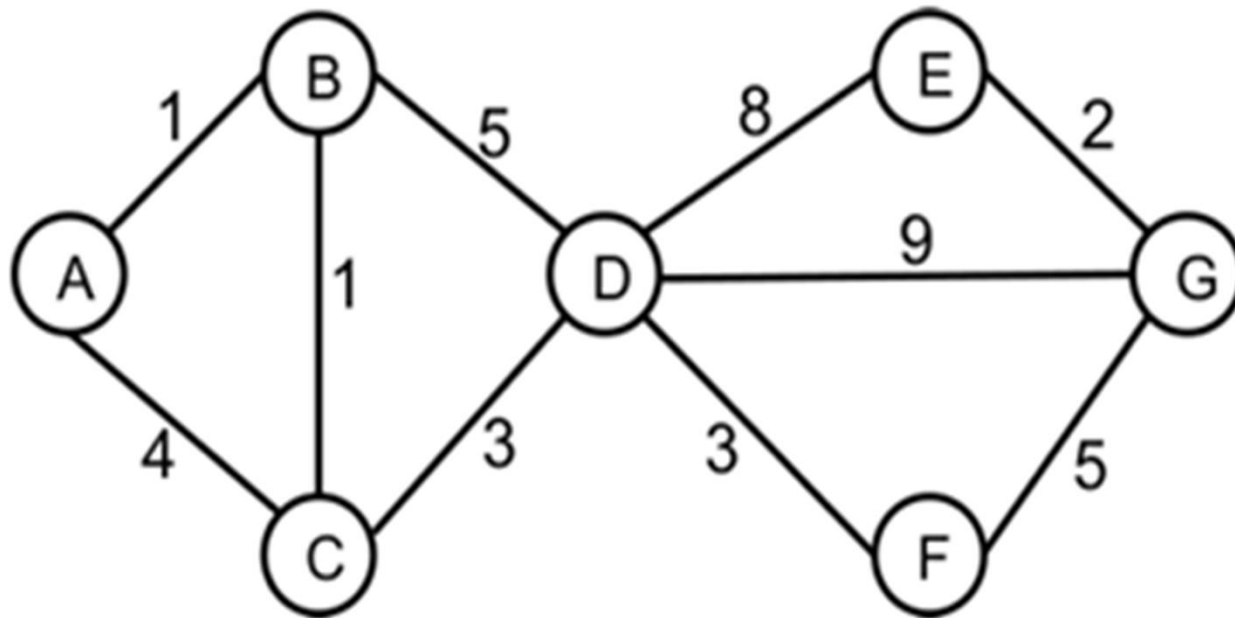
- Actions have varying costs
- Least cost solution is required

This is the only uninformed search that worries about costs.

- **Iterative-Deepening Search:**

- Space is limited and the shortest solution path is required

Homework: Apply Uninformed Search Methods



Apply BFS, DFS and UCS

Apply BFS, DFS and UCS

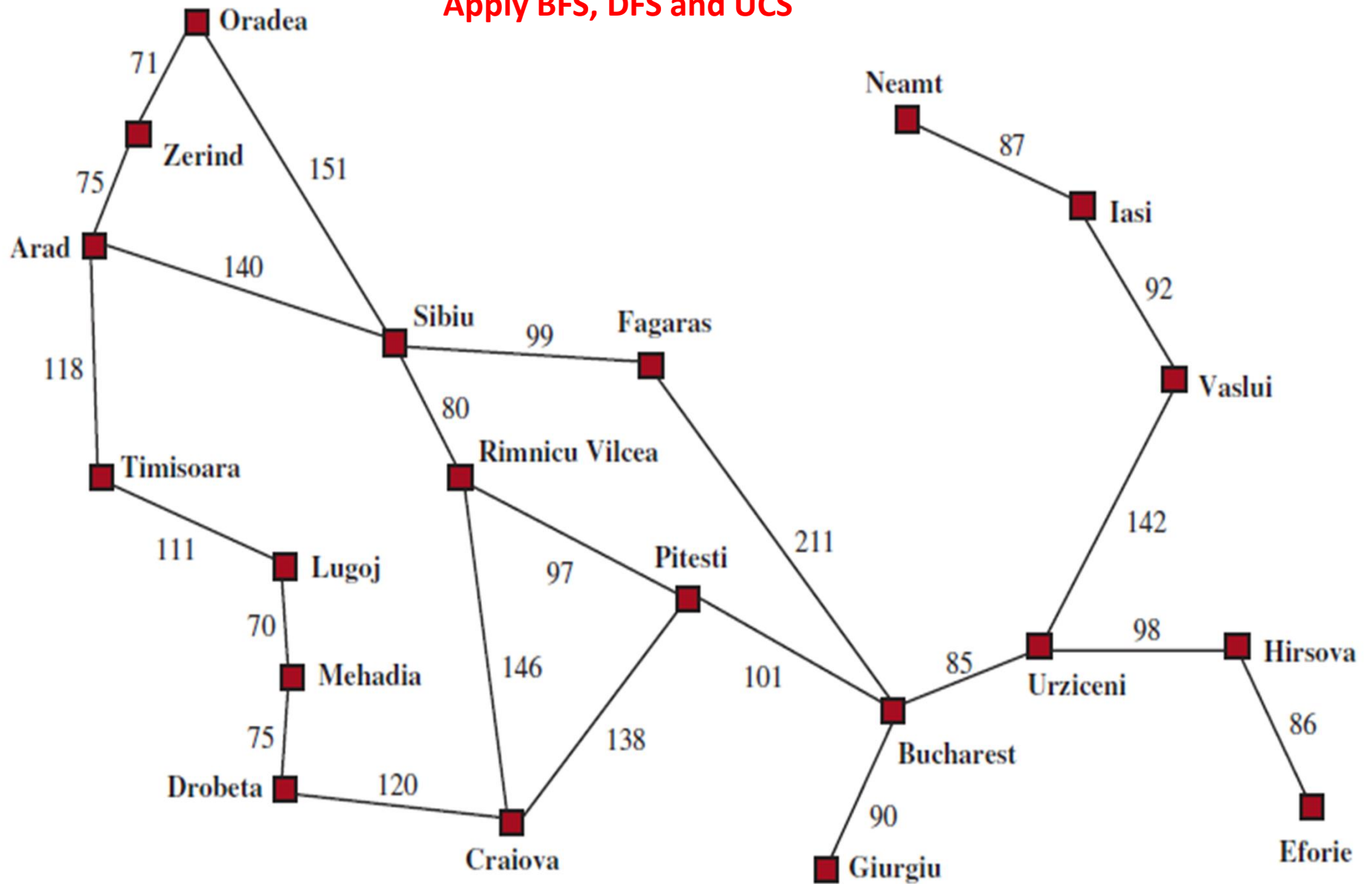


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Uniform Cost Search (UCS) Algorithm

1. Initialization:

- Initialize the priority queue PQ with the start node S and path cost 0.
- Initialize the explored set Explored as empty.
- Priority Queue: $PQ = \{(0, S)\}$

2. While PQ is not empty:

1. Remove the node n with the lowest path cost $g(n)$ from PQ.
2. If n is the goal node, return the path and cost.
3. If n is not in Explored, add n to Explored.
4. For each neighbor m of n :
 - Calculate the new path cost $g(m) = g(n) + \text{cost}(n, m)$.
 - If m is not in Explored or PQ, add m to PQ with $g(m)$.
 - If m is in PQ with a higher cost, update m in PQ with the new lower cost $g(m)$.